
DNN-FORWARDTESTING: A NEW TRADING STRATEGY VALIDATION USING STATISTICAL TIMESERIES ANALYSIS AND DEEP NEURAL NETWORKS

✉ **Ivan Letteri***

Department of Information Engineering, Computer Science and Maths
University of L'Aquila
L'Aquila, Italy, 67100
ivan.letteri@univaq.it

✉ **Giuseppe Della Penna**

Department of Information Engineering, Computer Science and Maths
University of L'Aquila
L'Aquila, Italy, 67100
giuseppe.dellapenna@univaq.it

✉ **Giovanni De Gasperis**

Department of Information Engineering, Computer Science and Maths
University of L'Aquila
L'Aquila, Italy, 67100
giovanni.degasperis@univaq.it

✉ **Abeer Dyoub**

Department of Information Engineering, Computer Science and Maths
University of L'Aquila
L'Aquila, Italy, 67100
abeer.dyoub@univaq.it

ABSTRACT

In general, traders test their trading strategies by applying them on the historical market data (*backtesting*), and then apply to the future trades the strategy that achieved the maximum profit on such past data.

In this paper, we propose a new trading strategy, called *DNN-forwardtesting*, that determines the strategy to apply by testing it on the possible future predicted by a deep neural network that has been designed to perform stock price forecasts and trained with the market historical data.

In order to generate such an historical dataset, we first perform an exploratory data analysis on a set of ten securities and, in particular, analyze their volatility through a novel k-means-based procedure. Then, we restrict the dataset to a small number of assets with the same volatility coefficient and use such data to train a deep feed-forward neural network that forecasts the prices for the next 30 days of open stocks market. Finally, our trading system calculates the most effective technical indicator by applying it to the DNNs predictions and uses such indicator to guide its trades.

The results confirm that neural networks outperform classical statistical techniques when performing such forecasts, and their predictions allow to select a trading strategy that, when applied to the real

*<https://www.ivanletteri.it>

future, increases Expectancy, Sharpe, Sortino, and Calmar ratios with respect to the strategy selected through traditional backtesting.

Keywords Statistical Learning · Deep Learning · Multi Layer Perceptron · kmeans · Technical Analysis · Stock Market Prediction · Trading System · Algorithmic Trading · Backtesting · Forwardtesting

Contents

1	Introduction	2
2	Preliminaries	3
2.1	Technical Analysis	3
2.2	Cumulative Returns	4
2.3	Volatility	4
2.4	Trading Strategies	6
2.5	Forecasting Error Metrics	7
2.6	Profit and Risk Metrics	7
3	Dataset Analysis	8
3.1	Volatility Analysis	8
3.2	ANF and EOG price time series	10
3.3	Synchrony between ANF and EOG stocks	11
4	Forecasting with Statistical Methods	12
4.1	ARIMA model	12
4.2	Prophet model	15
5	Forecasting with Deep Learning Methods	17
5.1	The Deep Neural Networks	17
5.2	DNN-Forwardtesting Experiment	19
5.2.1	Results	19
5.2.2	Baseline comparison	20
6	Conclusion	21

1 Introduction

Stock market forecasting is considered a research field with promising returns for traders and investors. However, there are considerable challenges to predicting stock market trends accurately and precisely enough due to their complexity, chaotic and non-linear nature. Indeed, traditional statistical models, which have been extensively applied to the market trend prediction so far, can easily handle only linear or stationary sequences.

According to the Efficient Market Hypothesis, stock prices reflect all past information, so the share prices themselves and the trading volume are sufficient to predict future price movements. Indeed, since new information is unpredictable and stock prices are adjusted immediately after the information is made public, the stock price follows a *random walk* Fama [1965] or, more generally, a stochastic process). Therefore, we first used traditional time series modelling techniques such as the Autoregressive Integrated Moving Average (ARIMA) model to predict stock prices. However, even if ARIMA model has its strength in robustness and efficiency in terms of *short-term* forecasting (Adebiyi et al.

[2014]), most ARIMA models are univariate, so they usually model only the historical stock price series (Lux and Kaizoji [2007]). On the other hand, in our work, we aim at an exhaustive prediction by means of price action trading activity (also called “*naked trading*”), i.e., considering daily information on maximum, minimum, opening, and closing prices and trading volume.

Artificial intelligence methods are being currently employed in a variety of tasks, for example to classify cyber attacks (e.g., Letteri et al. [2018], Marín et al. [2021]), predict network traffic anomalies (e.g., Letteri et al. [2019a], Letteri et al. [2019a]), and predict the course of a disease. Among such AI methods, artificial neural networks (ANN) and, in particular deep neural networks (DNN) proved suitable for dealing with complex nonlinear problems with multiple influencing factors (Letteri et al. [2022a])). Indeed, they are often used for image recognition and natural language processing (see, e.g., Soniya et al. [2015]), but are being applied also to the financial market, e.g., to predict stock prices using textual news analysis (Day and Lee [2016]). Actually, the experiments reported in this paper confirm that DNNs obtain the best overall accuracy in the price forecast task under consideration, even if they require more time to be tuned, if compared to two state-of-the-art statistical models such as ARIMA and Prophet.

In this paper, we propose a framework that uses stock prices historical data to train a set of DNNs to forecast the future (next month) stock prices. Such predictions are exploited in a novel way to determine the most profitable technical indicator(s) to be used as the basis of the trading strategy that is then executed by a robot advisor. Indeed, typically, traders test their (algorithmic trading) strategies, i.e., the technical indicators to watch and how to react to their values, on the historical market data (the so called *backtesting*), and then apply to the future trades the strategy that achieved the maximum profit on such past data. On the other hand, the term *forward testing* (also known as paper testing ²) is sometimes used in the literature to indicate a technique that updates the trading strategy in real time by looking at the current market data. In this paper we propose a “*forward testing in the future*”, that we shall call *DNN-forwardtesting* in the following, instead of the real time one. In such a technique, the best strategy is devised by looking at the profits earned by applying the candidate strategies directly to the possible future predicted by the DNN. Indeed, we leverage on the capabilities of the DNN to learn from the past trends and characteristics that would be difficult or even impossible to capture using the simple indicator-based analyses performed by the classical approach. In such sense, our forward testing is able to better exploit the available historical data and allow a finer strategy definition.

To verify our approach, we performed a model-based clustering using k-means++ on ten different stocks. Then we selected two shares, issued by companies operating in completely different sectors, from the cluster with the same medium volatility i.e., with price fluctuations that are not excessively large or small (as the ones, e.g., of a tech company stock or a large blue-chip company stock, respectively)

The experiments show that our technique allows the trader to choose a strategy that is more or equally profitable than the one that would be selected with traditional backtesting, if applied on the same historical data. Therefore, DNN-forwardtesting appears a better strategy selection criterion.

2 Preliminaries

This section describes some preliminary concepts related to trading, which are useful for a better understanding of the statistical processes adopted and the methodologies applied for trend price forecasting.

Specifically, the first part will list the formulations of concepts related to technical analysis, cumulative returns and price volatility. In the second part, the performance evaluation metrics of the models and the trading strategies applied.

2.1 Technical Analysis

Technical analysis (TA) represents the type of investment analysis that uses simple mathematical formulations or graphical representations of the time series of financial assets to explore trading opportunities. In its algorithmic form, TA uses the analysis of asset price history series (Wang et al. [2021]), defined as OHLC, i.e., the opening, highs, lowest and closing prices of an asset, typically represented with candlesticks charts (see, e.g. in fig. 1).

For each timeframe t , the OHLC of an asset is represented as a 4-dimensional vector $X_t = (x_t^{(o)}, x_t^{(h)}, x_t^{(l)}, x_t^{(c)})^T$, where $x_t^{(l)} > 0$, $x_t^{(l)} < x_t^{(h)}$ and $x_t^{(o)}, x_t^{(c)} \in [x_t^{(l)}, x_t^{(h)}]$.

²<https://www.investopedia.com/terms/p/papertrade.asp>

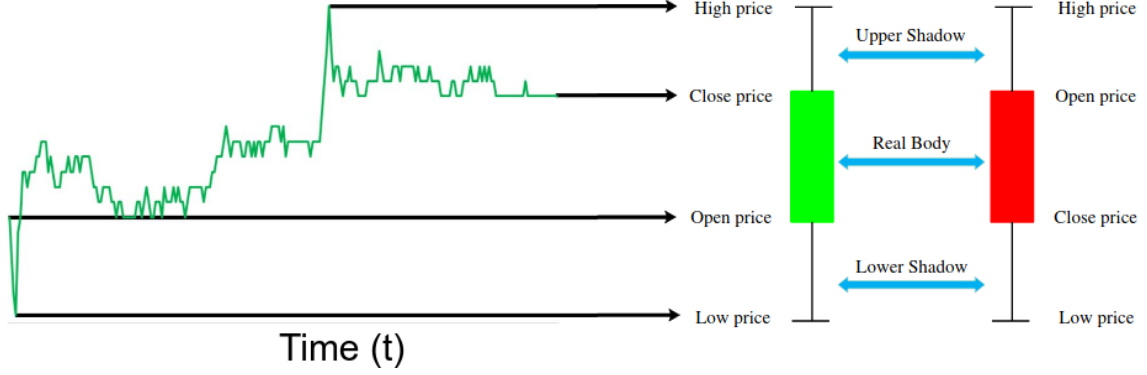


Figure 1: Example of candlestick chart.

2.2 Cumulative Returns

Given a time series TS , the return of the i -th trade is defined as: $r_i = \frac{TS(sell_i) - TS(buy_i)}{TS(buy_i)}$ where $TS(sell_i)$ and $TS(buy_i)$ denote the price of conducting the i -th *sell* and *buy* trades, respectively (see Yang et al. [2020]). Considering the transaction costs like *fees & slippage* (fs_i), a Cumulative Return ($CR(n)$) is the aggregate amount gained or lost over time with a set of trades, and it is calculated as follows:

$$CR(n) = \prod_{i=1}^n \left(1 + \frac{TS(sell_i) - TS(buy_i) - fs_i}{TS(buy_i)} \right)$$

where n is the number of trades, and fs_i the transaction costs in the i -th trade, which can be approximately considered as $fs_i \approx TS(s_i) \cdot k$, where k is the transaction cost rate.

It is worth noting that the return in each sub-period of the TS is rarely identical, so compounding it is impossible (as detailed in ³).

On the other hand, *logarithmic cumulative returns* measure the rate of exponential growth, instead of measuring the percent of price change for each sub-period. It is measured the exponent of its natural growth during that time, and then it is added each i sub-period of the exponential growth to get the total growth for the period, as follows:

$$\ln(CR(n)) = \sum_{i=1}^n \ln \left(1 + \frac{TS(sell_i) - TS(buy_i) - fs_i}{TS(buy_i)} \right).$$

Logarithmic cumulative returns present the following three properties that can not be observed in raw prices (Mikosch and Stărică [2000]):

1. The distribution of logarithmic cumulative returns is not normal, but rather with a high peak at zero and heavy-tailed. This comes from most days having little changes, i.e. a “flat” days.
2. There is dependence in the tails of the distribution. These tails represent major changes in returns, which do not happen often, but tend to trend together⁴.
3. The sample auto-covariance function (ACF) in the logarithmic return process is negligible at all lags, except possibly the first lag (Hoerlein [2017]).

For these reasons, in this work we focused on the logarithmic cumulative returns.

2.3 Volatility

While forecasting changes in stock returns is a very hard task, forecasting the size of changes (i.e. the volatility) seems more promising. Volatility quantifies the dispersion of returns. Unfortunately, this dispersion can not be measured and volatility is not directly observable, but it is possible to estimate it (Petneházi and Gáll [2018]).

³<https://www.ivanletteri.it/2022/06/29/why-lr-is-better-then-sr/>

⁴<https://digitalcommons.georgiasouthern.edu/etd/1409/>

Most finance textbooks use standard deviation (*StdDev*) as a measure of volatility of a stock price as follows:

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (\ln_{CR(n)}(x_t^{(c)}) - \ln_{CR(n)}(\bar{x}_t^{(c)}))^2$$

where $\ln_{CR(n)}(x_t^{(c)})$ are the logarithmic cumulative returns of the close price items, the $\ln_{CR(n)}(\bar{x}_t^{(c)})$ are the mean values of these observations, and N is the sample size.

However, the use of standard deviation loses information on the ordering of the price movements, grouping them around the average value of the returns (Sen et al. [2021]).

For this reason (and others detailed in ⁵), we focused on other formulations of historical volatility (HV) measures.

The following types of **Historical Volatility** measures are specific and efficient in the Quantitative Finance context (“*Measuring historical volatility*” - Colin Bennett⁶).

- The *Parkinson* (PK) estimator incorporates the stock’s daily *high* and *low* prices and is calculated as follow:

$$PK = \sqrt{\frac{1}{4N \ln_{CR(n)} 2} \sum_{i=1}^N (\ln_{CR(n)} \frac{x_t^{(h)}}{x_t^{(l)}})^2}.$$

PK provides completely separate information from using time-based sampling such as closing prices, but it cannot handle trends and jumps, so it systematically underestimates the volatility.

- The *Garman-Klass* (GK) estimator incorporates open, low, high, and close prices and is calculated as follows:

$$GK = \sqrt{\frac{1}{N} \sum_{i=1}^N \frac{1}{2} (\ln_{CR(n)} \frac{x_t^{(h)}}{x_t^{(l)}})^2 - \frac{1}{N} \sum_{i=1}^N (2 \ln_{CR(n)} 2 - 1) (\ln_{CR(n)} \frac{x_t^{(c)}}{x_t^{(o)}})^2}.$$

This method is robust for opening jumps in price and trend movements. However, it takes into account not only the price at the beginning and end of the time interval but also intraday price extremums.

- The *Rogers-Satchell* (RS) estimator, unlike the PK and GK estimators, RS incorporates drift term (mean return not equal to zero). It is calculated as follows:

$$RS = \sqrt{\frac{1}{N} \sum_{t=1}^N (\ln_{CR(n)}(\frac{x_t^{(h)}}{x_t^{(c)}}) \ln_{CR(n)}(\frac{x_t^{(h)}}{x_t^{(o)}}) + \ln_{CR(n)}(\frac{x_t^{(l)}}{x_t^{(c)}}) \ln_{CR(n)}(\frac{x_t^{(l)}}{x_t^{(o)}}))}.$$

However, it does not take into account price movements between trading sessions, and underestimate volatility since price jumps periodically occur between sessions.

- The *Yang-Zhang* (YZ) estimator Yang and Zhang [2000] incorporates open, low, high, and close prices, and is calculated as follows:

$$\begin{aligned} \sigma_{OpenToCloseVol}^2 &= \frac{1}{N-1} \sum_{i=1}^N (\ln_{CR(n)} \frac{x_t^{(c)}}{x_t^{(o)}} - \ln_{CR(n)} \frac{x_t^{(o)}}{x_t^{(c)}})^2 \\ \sigma_{OvernightVol}^2 &= \frac{1}{N-1} \sum_{i=1}^N (\ln_{CR(n)} \frac{x_t^{(o)}}{x_{t-1}^{(c)}} - \ln_{CR(n)} \frac{x_t^{(c)}}{x_{t-1}^{(o)}})^2 \\ YZ &= \sqrt{\sigma_{OvernightVol}^2 + k \sigma_{OpenToCloseVol}^2 + (1-k) \sigma_{RS}^2} \end{aligned}$$

where $k = \frac{0.34}{1.34 + \frac{N+1}{N-1}}$.

YZ is considered the most powerful volatility estimator. It has the minimum estimation error since it is a weighted average of RS, the close-open volatility and the open-close volatility.

⁵<https://www.ivanletteri.it/2022/06/29/why-stddev-is-not-a-good-measure-of-volatility/>

⁶https://dynamiprject.files.wordpress.com/2016/01/measuring_historic_volatility.pdf

2.4 Trading Strategies

In this section, we discuss two distinct trading strategy classes used in our framework: *trend following* and *mean reversion*.

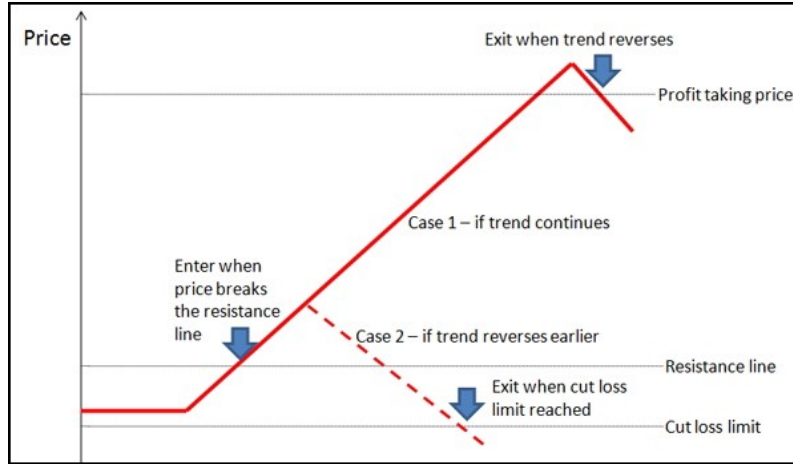


Figure 2: Trend Following Strategy taken from trendfollowing.com

Trend Following. One way to trade a trend is to look at an asset with a resistance line (see fig. 2). Once the price breaks through resistance, a trader places an order in the direction of the breakout⁷.

Trend-following, or momentum, strategies have the attractive property of generating trading returns with a positively skewed statistical distribution. Consequently, they tend to hold on to their profits and are unlikely to have severe ‘drawdowns’ (Martin [2021]).

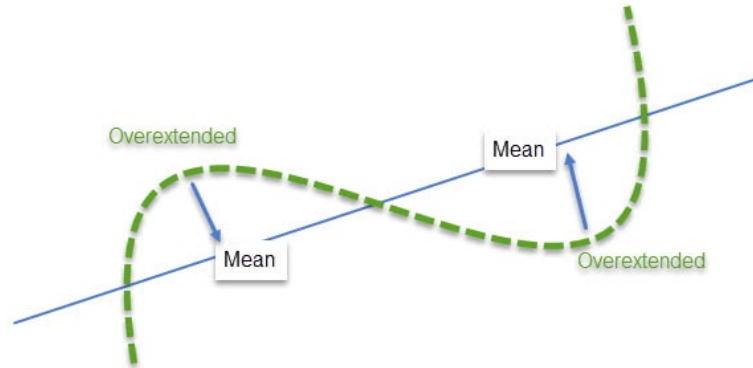


Figure 3: Mean Reversion Strategy description from tradergav.com/what-is-mean-reversion-trading-strategy/

Mean Reversion. The idea of mean reversion strategies is that the maximum and minimum price of a security is temporary, and that it will tend to the average over time (see Huang and Wang [2016]).

Fig. 3 shows how the mean reversion is applied by first identifying the stock’s trading range and then calculating the average price using analytical techniques including volatility (3.1).

Elliott et al. [2005] explains how mean-reverting processes might be used in pairs trading and developed several methods for parameter estimation. Avellaneda and Lee [2010] used mean-reverting processes for pairs trading, and modeled the hitting time to find the exit rule of the trade.

⁷[https://en.wikipedia.org/wiki/Breakout_\(technical_analysis\)](https://en.wikipedia.org/wiki/Breakout_(technical_analysis))

It is worth noting that trend following and mean reversion strategies, although theoretically opposing ideas, are not in conflict with each other and they are therefore both applicable at the same time to the same security. Obviously, it is not possible to apply them to the same time window in order to make money with both, but there is no reason why they are incompatible with each other. Trends, in fact, tend to occur on longer time horizons, while reversals, which are in fact small swings around these trends, on shorter ones. Therefore, in our tests we used them both alternatively.

2.5 Forecasting Error Metrics

In this section we introduce the different metrics used to estimate the forecasting error of the models presented in the paper.

- *Mean Square Error* (MSE): measures the average of the squared difference between the correct and predicted values (called prediction *error* or *residual*) as follows:

$$MSE(y, \hat{y}) = \sum_{t=0}^N (y_t - \hat{y}_t)^2$$

- *Root Mean Square Error* (RMSE): is the standard deviation of the residuals (prediction errors), i.e, how spread out these residuals are, as follows:

$$RMSE(y, \hat{y}) = \sqrt{\sum_{t=0}^N (y_t - \hat{y}_t)^2}.$$

- *Mean Absolute Error* (MAE): measures the average of the absolute differences between the correct and predicted values as follows:

$$MAE(y, \hat{y}) = \frac{1}{N} \sum_{t=0}^N \left| \frac{y_t - \hat{y}_t}{y_t} \right|$$

where n is the number of fitted points, y is the actual value, $\hat{y}$ is the forecast value.

- *Mean Absolute Percentage Error* (MAPE): measures the accuracy as a percentage, and can be calculated as the average absolute percent error for each time period minus the actual values divided by the actual values. MAPE is the most common measure used to forecast error, and works best if there are no extremes to the data (and no zeros).
- *Explained Variance Regression Score* (EVS): measures the error dispersion as follows:

$$EVS(y, \hat{y}) = 1 - \frac{Var(\hat{y} - y)}{Var(y)}$$

where the Var is biased variance, (i.e. $Var(\hat{y} - y) = \frac{1}{n} \sum (error - mean(error))$).

2.6 Profit and Risk Metrics

In this section, we set out the metrics related to downside risk by estimating the potential loss in value of the stock.

- The *Maximum drawdown* (MDD)⁸ measures the largest decline from the peak in the whole trading period, to show the worst case, as follows: $MDD = \max_{\tau \in (0, t)} [\max_{t \in (0, \tau)} \frac{n_t - n_\tau}{n_t}]$.
- The *Sharpe ratio* (SR)⁹ is a risk-adjusted profit measure, which refers to the return per unit of deviation as follows: $SR = \frac{\mathbb{E}[r]}{[\sigma]}$.
- The *Sortino ratio* (SoR)¹⁰ is a variant of the risk-adjusted profit measure, which applies DD as risk measure: $SoR = \frac{\mathbb{E}[r]}{DD}$.

⁸<https://www.investopedia.com/terms/m/maximum-drawdown-mdd.asp>

⁹<https://www.investopedia.com/terms/s/sharperatio.asp>

¹⁰<https://www.investopedia.com/terms/s/sortinoratio.asp>

- The *Calmar ratio* (CR)¹¹ is another variant of the risk-adjusted profit measure, which applies MDD as risk measure: $CR = \frac{\mathbb{E}[r]}{MDD}$.

To check the goodness of trades, we mainly focused on the *Total Returns* $R_k(t)$ for each stock ($k = 1, \dots, p$) in the time interval ($t = 1, \dots, n$)

$$R_k(t) = \frac{Z_k(t + \Delta t) - Z_k(t)}{Z_k(t)} \quad (1)$$

and furthermore analysing the standardized returns $r_k = (R_k - \mu_k)/\sigma_k$, with ($k = 1, \dots, p$), where σ_k is the standard deviation of R_k , μ_k denote the average overtime for the studied period.

3 Dataset Analysis

This section describes the methodology through which the final dataset has been collected for the experiment, the criteria adopted for the statistical analysis of the data, the training process and the tests conducted on the DNNs with optimized processes (Letteri et al. [2020a]).

We started by analysing ten stocks picked randomly among the securities listed on the New York Stock Exchange (NYSE) shown in tab. 1.

Table 1: List of 10 stocks randomly selected [source `it.finance.yahoo.com`].

Ticker	Company	Market
CSGKF	Credit Suisse Group AG	Other OTC
EOG	EOG Resources, Inc.	NYSE
META	Meta Platforms, Inc.	Nasdaq GS
NKE	NIKE, Inc.	NYSE
DIS	The Walt Disney Company	NYSE
PG	The Procter & Gamble Company	NYSE
QQQ	Invesco QQQ Trust	Nasdaq GM
IBM	International Business Machines Corporation	NYSE
ANF	Abercrombie & Fitch Co.	NYSE
CS	Credit Suisse Group AG	NYSE

Technically speaking, the dataset used in this paper consists of the time series of OHLC prices, over the time period from 30 October, 2011 to 30 November, 2021, for a total period of 2537 open market days. OHLC prices are the opening, highest, lowest and closing prices of an asset, and are commonly used to analyze the assets price history when performing the so called technical analysis (see sect. 2.1) to explore trading opportunities.

3.1 Volatility Analysis

Some stocks are more volatile than others. For example, the shares of a large blue-chip company may not have large price fluctuations and are therefore said to have *low volatility*, whereas the shares of a tech stock which fluctuate often having *high volatility*. There are also stocks with *medium volatility*¹², and this is the case of the assets that we decided to select in this paper.

Therefore, in order to filter stocks with medium volatility, we first created a dataset composed by the time series of the PK, GK, RS, YZ historical volatility estimators for our securities. The data has been standardized and clustered with K-means++ Arthur and Vassilvitskii [2007].

The procedure adopted to find the optimal number of clusters is the Elbow method¹³, i.e.:

1. Compute k-means clustering for different values of k . In our case, we varied k from 2 to 20 clusters.
2. For each k , is calculated the total within-cluster sum of squares (wss).

¹¹<https://www.investopedia.com/terms/c/calmarratio.asp>

¹²<https://www.fool.com/investing/how-to-invest/stocks/stock-market-volatility/>

¹³<https://www.ivanletteri.it/2022/10/11/optimal-volatility-clusters/>

3. Plot the curve of wss according to the number of clusters k .
4. Find the location of a bend (knee) in the plot, which is generally considered as indicator of the appropriate number of clusters.

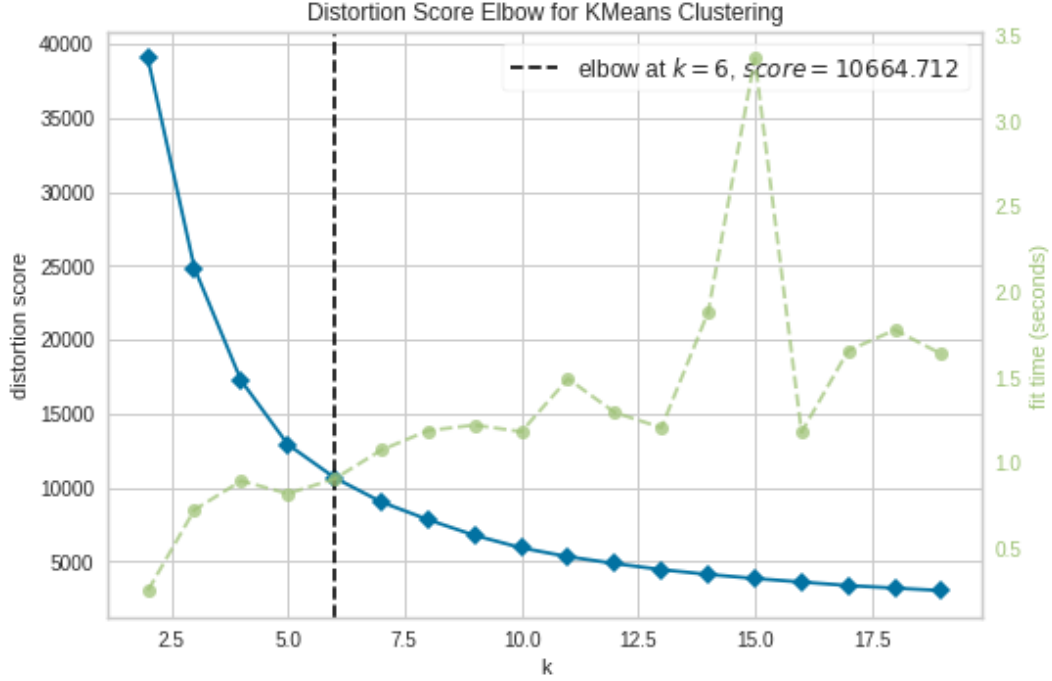


Figure 4: Elbow method to determine optimal cluster in $k = 6$

Fig. 4 shows that the best clustering is for $k = 6$, and fig. 5 clearly shows that, between the considered stocks, ANF and EOG have all the types of observations spread over all the k-means clusters for each HV estimator considered. This makes them good candidates for our final dataset because they gather all the characteristics (pros and cons) of the four history volatility estimators adopted.

Table 2: Volatility measures of the HV estimators.

HV Est.	ANF Volatility Measures			
	Min	Max	Mean	StdDev
PR	-0.264694	0.344451	0.000448	0.034559

PK	0.205809	1.061728	0.401217	0.116944
GK	0.204787	1.128780	0.406463	0.121297
RS	0.203886	1.146890	0.409334	0.123215
YZ	0.236222	1.521485	0.527025	0.194645

HV Est.	EOG Volatility Measures			
	Min	Max	Mean	StdDev
PR	-0.320072	0.165702	0.000625	0.024650

PK	0.136258	1.096084	0.290084	0.122853
GK	0.145601	1.188479	0.291579	0.128462
RS	0.148818	1.266812	0.292796	0.134589
YZ	0.169166	1.952395	0.366873	0.210179

Black [1976] observed that the volatility tends to increase when the price drops and appears a significant skew in the volatility. As shown in tab. 2, we noted that the percentage returns (PR) respect to the volatility measures, provided by all the estimators, consistently suggests that it may be more useful in estimating the market volatility for short-term trading purposes rather than characterizing the evolution of the historical volatility over the long term.

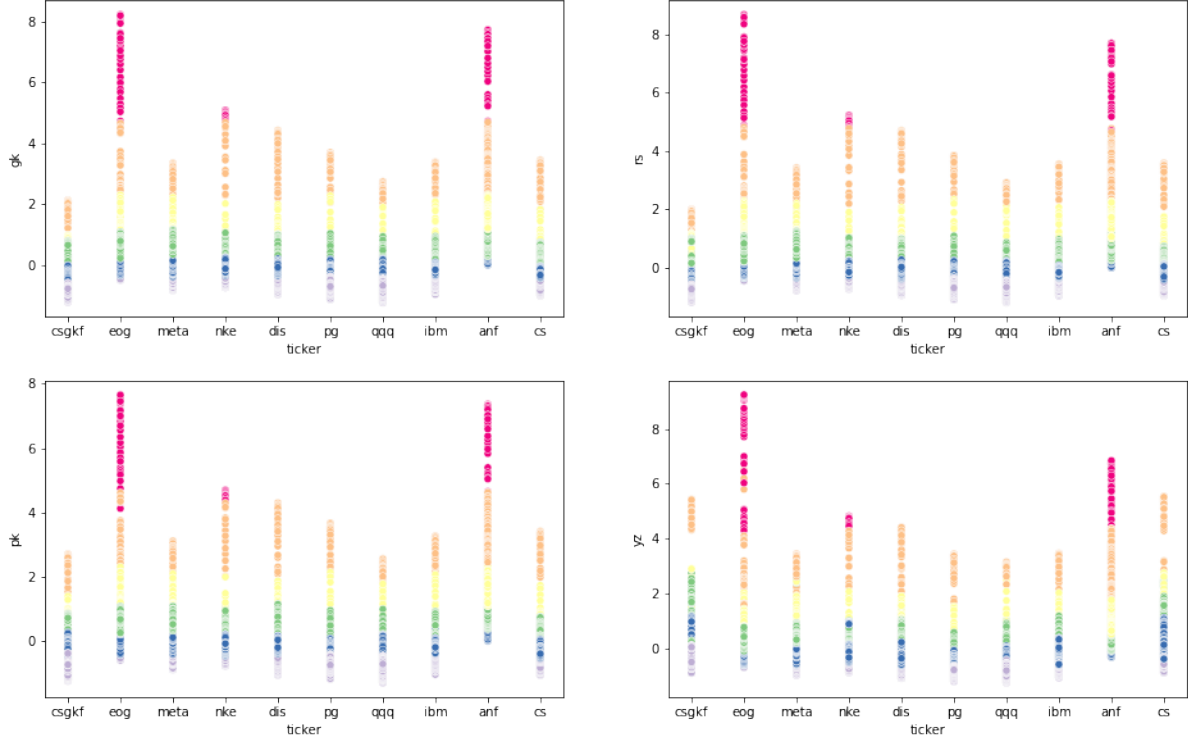


Figure 5: Kmeans++ Clusters with $k = 6$ of the Historical Volatility estimators dataset.

3.2 ANF and EOG price time series

ANF was founded at the end of September 1996, and from April to October 2011 it was several times close to the all-time high, always encountering resistance.

On November 23, 2021 the company CEO announced net sales of \$905 million, up 10% as compared to the previous year and up 5% as compared to the pre-COVID 2019 third quarter net sales (source GlobeNewswire¹⁴). Fig. 6 shows the ANF stock price trends starting from October 30th, 2011 to November 30th, 2021. On the other hand, the EOG

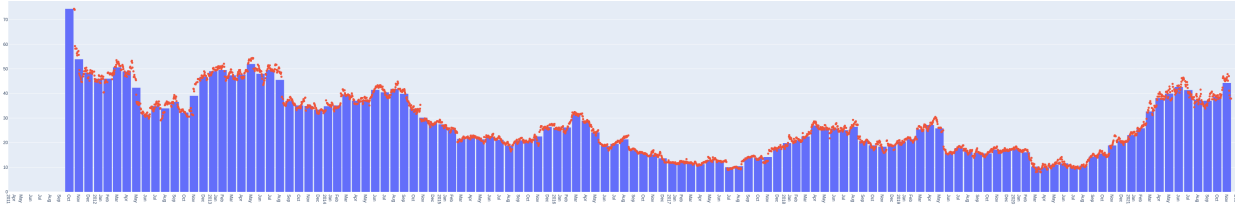


Figure 6: ANF average trend from October 30th, 2011 to November 30th, 2021

stock, with a market value of \$55.21 billion and 84.08% institutional ownership, has gained 9.39% so far (source investopedia¹⁵). The company is expected to post quarterly earnings of \$3.24 per share in its next report. Fig. 7 shows the EOG stock price trends in the same time interval used for ANF.

ANF and EOG are certainly assets with a sometimes controversial trend and consequently well profitable if rightly analyzed, especially in the particular period of 2020/2021 due to the global pandemic. However, in the analysed period,

¹⁴www.globenewswire.com/news-release/2021/11/23/2339734/0/en/Abercrombie-Fitch-Co-Reports-Third-Quarter-Results.html

¹⁵<https://www.investopedia.com/3-oil-and-gas-stocks-to-watch-this-week-4628357>

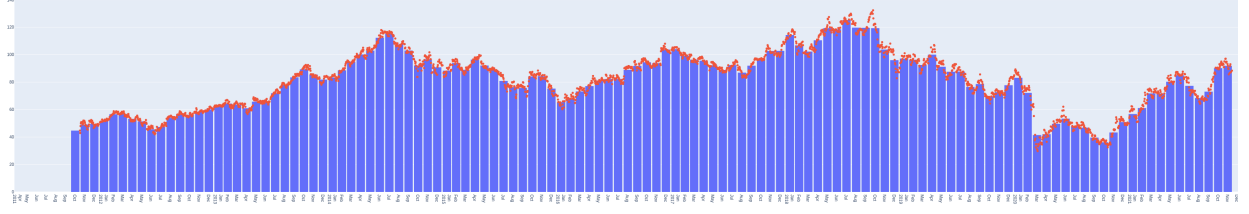


Figure 7: EOG average trend from October 30th, 2011 to November 30th, 2021

ANF and EOG do not show as the classic always profitable stocks (e.g. Tesla, Apple, Microsoft, or Bitcoin) to which trivially apply, in that period, a passive Buy and Hold strategy¹⁶ for gains.

The time series of price observations can be downloaded from Yahoo Finance¹⁷ and ¹⁸, or via our github repository¹⁹ also contains a copy of such data preprocessed and split into train and test sets to be used in a deep neural network.

3.3 Synchrony between ANF and EOG stocks

In order to prove that the dataset is general enough to model a variety of different shares with more or less the same volatility coefficient (vc), we also have to prove that the price time series corresponding to the selected assets are completely uncorrelated, i.e., ANF and EOG does not influence each other.

Therefore, after scaling the values with a *minMax* normalization ($x_i = \frac{x - x_{min}}{x_{max} - x_{min}}$), we evaluated the *synchrony* between the two financial assets using the Pearson coefficient and the Dynamic Time Warping.

The Pearson coefficient measures the linear relation between two continuous signals, and is defined as $r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}$, where x_i and y_i are, in our case, the close prices of the ANF and EOG stocks.

In fig. 9(b), the coefficients -1 and 1 indicate a perfect negative and positive correlation, respectively, whereas 0 stands for no correlation. It is worth noting that the Pearson coefficient is highly sensible to outliers, which can sensibly alter the correlation estimation, and requires the compared time series to contain homoscedastic data, having an homogeneous variance in the observation interval. However, both the considered time series do not contain many outliers and can be considered homoscedastic thanks to the medium volatility of the considered stock prices.

The overall Pearson coefficient between ANF and EOG is 0.28, which confirms that the two stocks are almost completely uncorrelated. However, this is a measure of the *global synchrony* in the overall period.

Therefore, for sake of completeness, we calculated the *local synchrony* in a small portions of the timespan by repeating the process along a moving window of 120 samples. Fig. 8(b) plots such moment-by-moment synchrony curve.

The Dynamic Time Warping (DTW) algorithm outperforms the Pearson correlation in detecting atypical functional dependencies (Linke et al. [2020]) between time series, even if they have a different number of samples. It calculates the optimal match between the two series by minimising the Euclidean distance between pairs of samples at the same time.

In Fig. 9, we can see the optimal match between the closing price of the ANF and EOG assets. The minimum path cost is $d = 209.95$, and such a large distance between the two stocks supports our hypothesis of a complete absence of influence between them.

¹⁶<https://www.investopedia.com/terms/b/buyandhold.asp>

¹⁷<https://it.finance.yahoo.com/quote/ANF?p=ANF&.tsrc>

¹⁸<https://it.finance.yahoo.com/quote/EOG?p=EOG&.tsrc>

¹⁹<https://github.com/IvanLetteri/DNN-ForwardTesting>

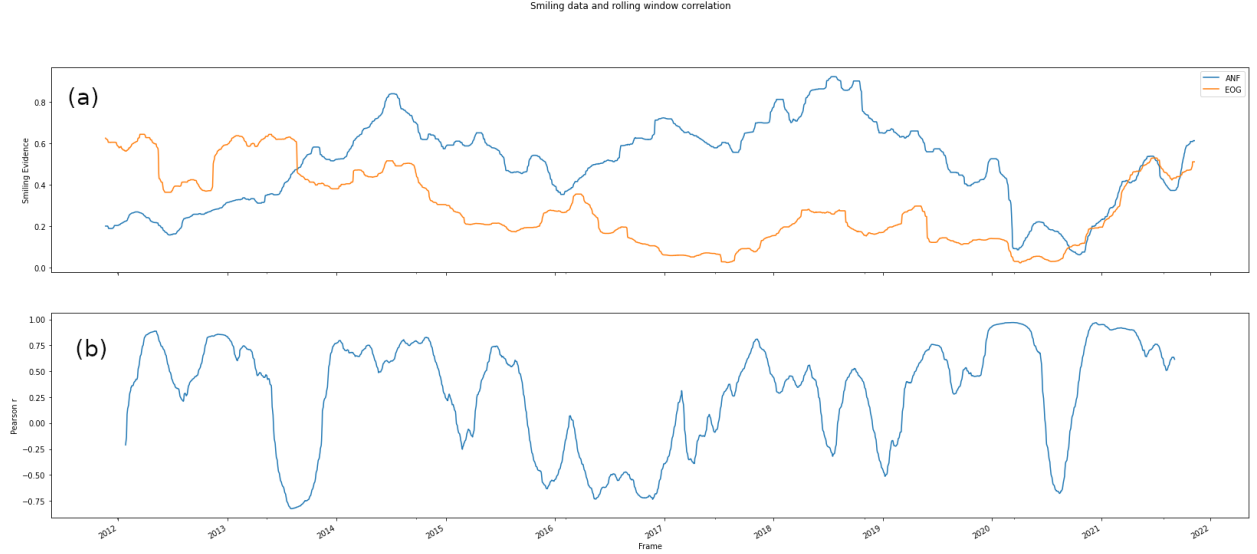


Figure 8: Figure (a) show smiling data between ANF and EOG, figure (b) the correlation coefficients.

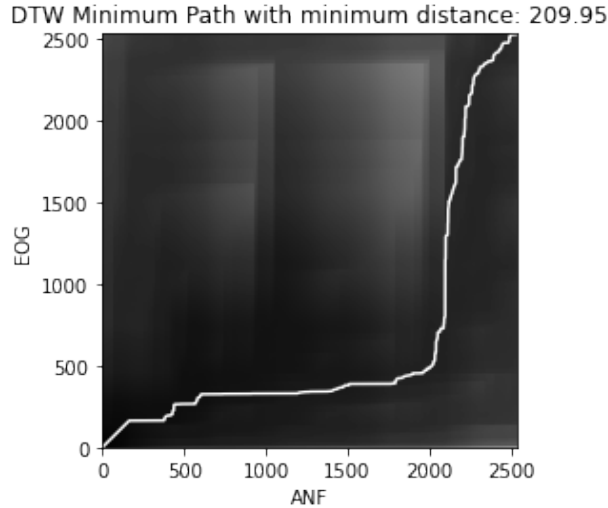


Figure 9: DTW Minimum Path with minimum distance between ANF and EOG close prices.

4 Forecasting with Statistical Methods

As a baseline for the proposed DNN-based forecast process, in this section we evaluate the performances of two well-known statistical time series forecasting methods, i.e., the ARIMA autoregressive model and the Prophet procedure, applied to our dataset.

4.1 ARIMA model

In this work, we applied a non-seasonal ARIMA models, neglecting the modulation effects of holidays and using therefore pure trend lines.

In general, an ARIMA model needs three parameters to run: the number of autoregressive terms p , the number of nonseasonal differences needed for stationarity d , and the number of lagged forecast errors in the prediction equation q (see, e.g., Marquez [1995] for more information on the meaning of such parameters).

As a first step, we checked the stationarity properties of the time series performing the Augmented Dikey Fuller (ADF) unit root test using the built-in method in the *statsmodels* Python package.

Table 3: ADF test stationarity with AIC optimization.

	Test Statistic	p-value	Lags	Observations	Critical Value		
					1%	5%	10%
<i>ANF</i>	-2.302	0.171	5	2529	-3.432	-2.863	-2.567
<i>EOG</i>	-2.422	0.135	5	2529	-3.433	-2.862	-2.567

Tab. 3 shows the number of lags considered, automatically selected based on the Akaike Information Criterion (AIC) (see Akaike [1974]) on ANF and EOG stock prices. The p-value results above the threshold (such as 5% or 1%) and suggests rejecting the null hypothesis, so the time series turns out not to be stationary, so the *Volatility* is not dependent on time/trend.

The results achieved exposed in tab. 4 are qualitatively similar for all the two assets selected: for the $\log(p_t)$ time series one cannot reject the null hypothesis of the presence of a unit root, signalling the non-stationarity of the series but with a first differencing the series, i.e. considering $\Delta \log(p_t) = \log(p_t) - \log(p_{t-1})$, makes it stationary. Thus the $\log(p_t)$ series are integrated of order one, and accordingly the models we considered are $ARIMA(p, 1, q)$.

In order to have a rough indication on the AR orders p , and on the MA orders q , we computed the sample autocorrelation function (ACF) and the partial autocorrelation function (PACF) for $\Delta \log(p_t)$ where:

- for an exact $MA(q)$, the ACF is zero when lags larger than q ;
- for an exact $AR(p)$, the PACF is zero when lags larger than p .

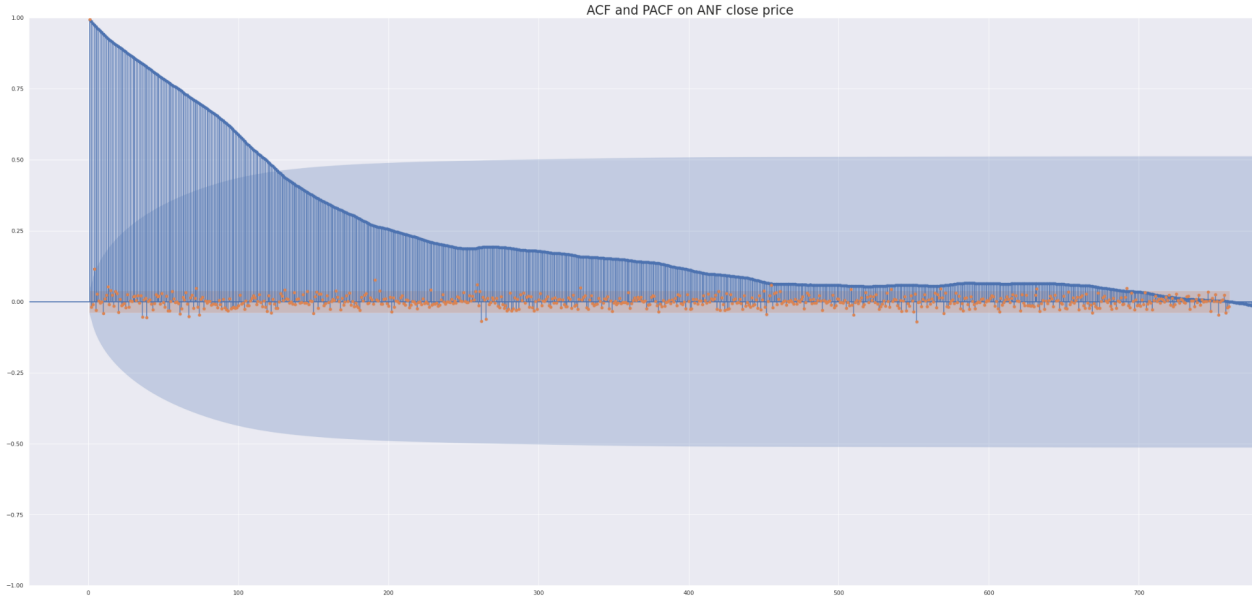


Figure 10: ACF and PACF of the ANF stock closing price.

In our case, we can see from the ACF plot in Fig. 10, the plot of lag values on the x-axis and how they increase, and at the same time the correlation between price and lagged price on the y-axis deteriorates to lag = 750 for ANF and lag = 330 for EOG (see fig. 12). This means that the values of the historical series of ANF are strongly correlated with those of the lagged series for only an initial period, then the correlation decreases faster and faster, especially in case of EOG stock.

One of the most widely used algorithms is the `auto.arima()` function developed for automatic time series forecasting with ARIMA models (Hyndman and Khandakar [2008]). More precisely, we used the Auto-ARIMA model implemented in the *pmdarima*²⁰ library, it discovers the optimal parameters trying various sets of p and q to minimize the selected

²⁰<https://alkaline-ml.com/pmdarima/setup.html>

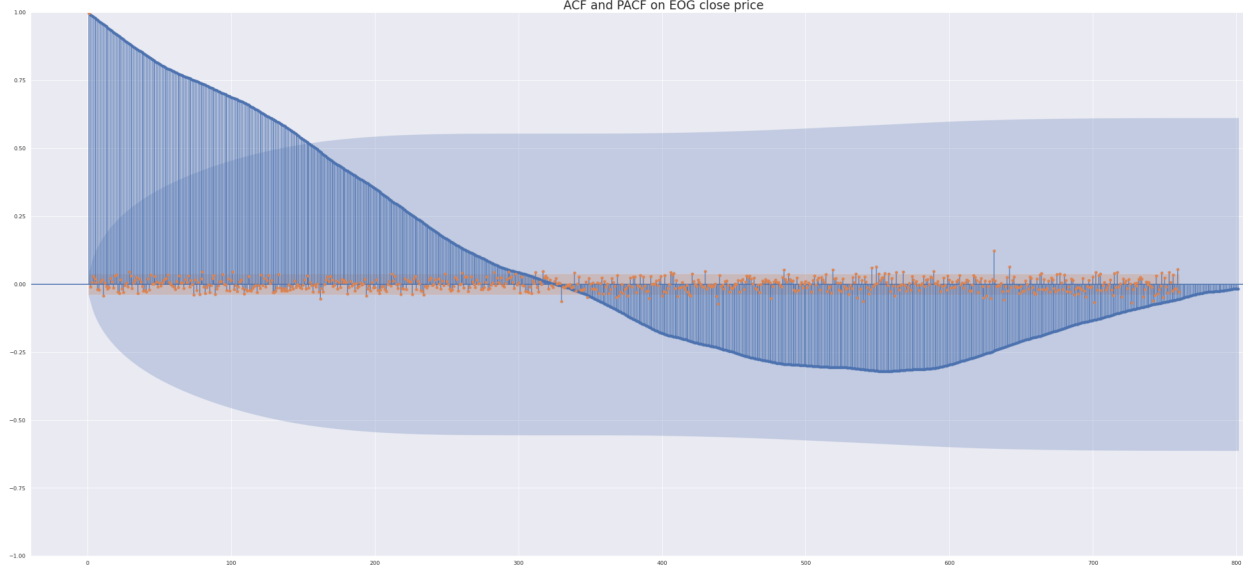


Figure 11: ACF and PACF of the EOG stock closing price.

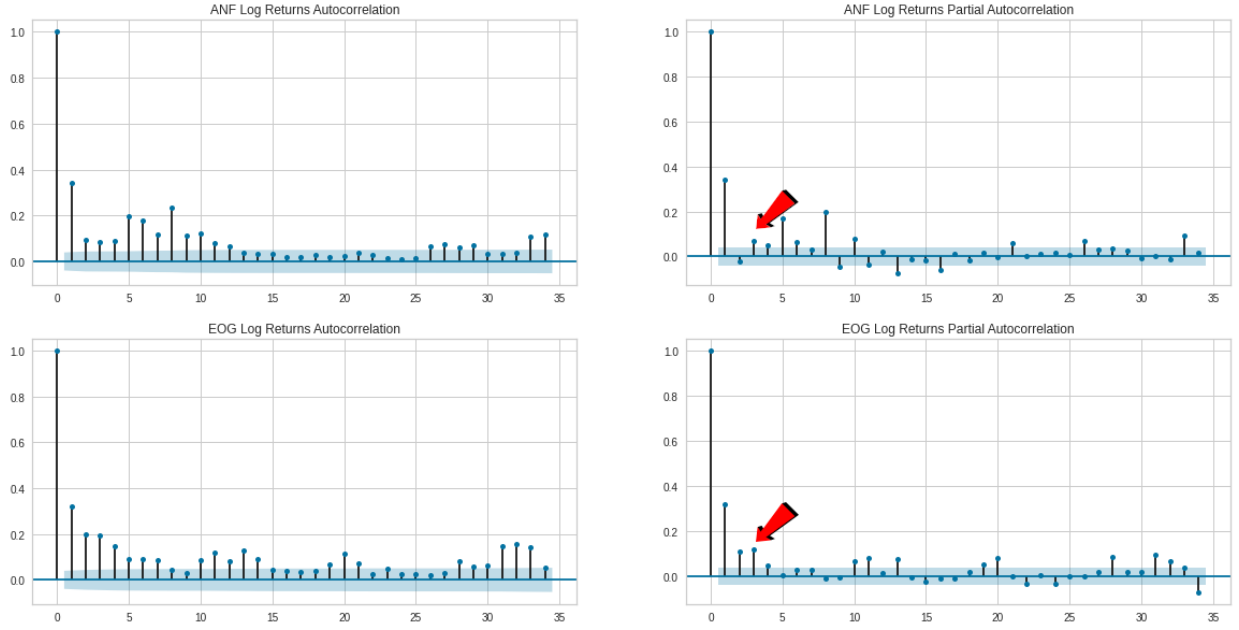


Figure 12: ACF and PACF of the ANF and EOG stock closing prices.

criterion that is, in our case, AIC, since it provides a good trade-off between the model fitting and the evaluation simplicity (Stoica and Selen [2004]) and also deals with the risk of overfitting and underfitting. The lower is the AIC value, the better is the result.

Despite that this algorithm allow us to implement the order selection process with relative ease in a standalone computer for short time series, efficient ARIMA model selection for ultra-long time series like our dataset is challenging, also with modern distributed computational environments like Google Colab. So, from the results of ACF and PACF shown in figs. 12, in order to optimize we established a range to analyze for the p value from 0 to 5 and for the q value from 0 to 2.

Table 4: ARIMA (p,d,q) models of ANF and EOG performing stepwise search to minimize AIC.

ARIMA (p,d,q)	ANF AIC	ARIMA (p,d,q)	EOG AIC
(0,1,0)	8323.575	(0,1,0)	10369.768
(1,1,0)	7652.108	(1,1,0)	10371.676
(0,1,1)	7294.347	(0,1,1)	10371.541
(1,1,1)	8317.976	(1,1,1)	10373.750

Table 4 shows that the best ARIMA model for ANF has $p = 0$, $d = 1$, and $q = 1$ also known as *simple exponential smoothing* (SES) model²¹ which corrects autocorrelated errors. It is worth note that, for some nonstationary time series (e.g., ones that exhibit noisy fluctuations around a slowly-varying mean), the random walk model does not perform as well as a moving average of past values. In other words, rather than taking the most recent observation as the forecast of the next observation, it is better to use an average of the last few observations in order to filter out the noise and more accurately estimate the local mean. The SES model uses an exponentially weighted moving average of past values to achieve this effect. EOG has $p = 0$, $d = 1$, and $q = 0$, also known as *random walk*²² (Danyliv et al. [2019]), where $y_{t+1} = y_t + \epsilon_t$, and ϵ_t are a sequence of centered, uncorrelated random variables. An ARIMA(0, 1, 0) series, when differenced once ($d = 1$), becomes an ARMA(0, 0), which is random, uncorrelated, and noise. If $X_1, X_2, X_3, \dots$ are the random variables in the series, this means that $X_{i+1} - X_i = \epsilon_{i+1}$ where $\epsilon_1, \epsilon_2, \dots$ are a sequence of centered, uncorrelated random variables, so $X_{i+1} = X_i + \epsilon_i$ reveals that we have a *Random Walk*.

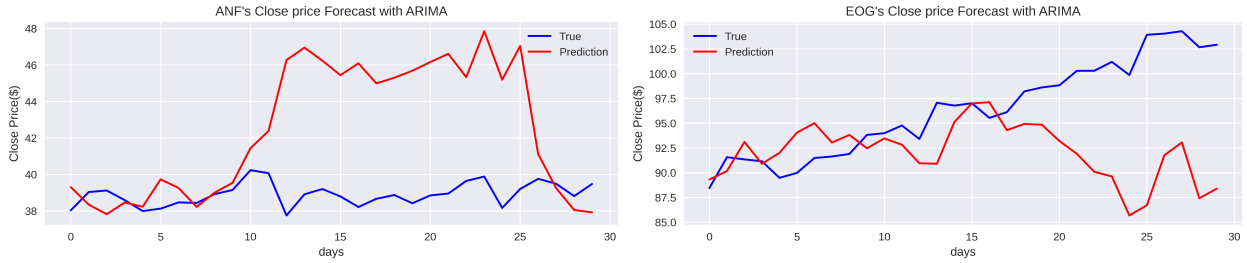


Figure 13: Detail of ARIMA forecast for the last 30 days of the ANF (left) and EOG (right) stock closing price.

Fig. 13 shows the predictions on the closing prices made with such auto-selected optimal ARIMA model for $n = 30$ days following the training timespan, which corresponds to the 2507 days of market from October 30, 2011 to October 16, 2021. Note that, in the following, for the sake of brevity we will omit the graphs and values of low, high, and open prices, since the forecast errors are always very similar between the four OLHC components.

Table 5: Error metrics of ARIMA on ANF and EOG stock price prediction.

	ARIMA				
	MSE	RMSE	MAE	MAPE	EVS
ANF	25.49	5.05	3.86	0.09	-0.02
EOG	56.23	7.50	5.42	0.06	-3.94

Table 5 reports very high error metrics. It is worth noting that the model performs a bit better with the EOG stock, but not enough to be considered as a suitable tool for forecasting.

4.2 Prophet model

Looking for a statistical method to improve the ARIMA's results, we turned to Facebook Prophet (see Žunić et al. [2020]). Prophet is "a procedure for forecasting time series based on an additive model where non-linear trends are fit with yearly, weekly, and daily seasonality, plus holiday effects. It works best with time series that have strong seasonal effects and several seasons of historical data. Prophet is robust to missing data and shifts in the trend, and typically handles outliers well." Facebook Open Source [2022].

²¹<https://people.duke.edu/~rnau/411arim.htm#ses>

²²<https://people.duke.edu/~rnau/411arim.htm#arima010>

More technically, Prophet is an additive regressive model which uses a time series with three main components: trend, seasonality, and holidays, combined in the equation $y(t) = g(t) + s(t) + h(t) + \epsilon(t)$, where $g(t)$ is the trend function which models non-periodic changes in the value of the time series, $s(t)$ represents the seasonality, i.e., periodic changes (e.g., the number of trades might also depend on the month/year), $h(t)$ represents the effects of holidays, which have a clear impact on most business time series, and $\epsilon(t)$ is the error term, following a normal distribution.

In our experiments, we used Prophet "out of the box", leaving all the default parameter selections.

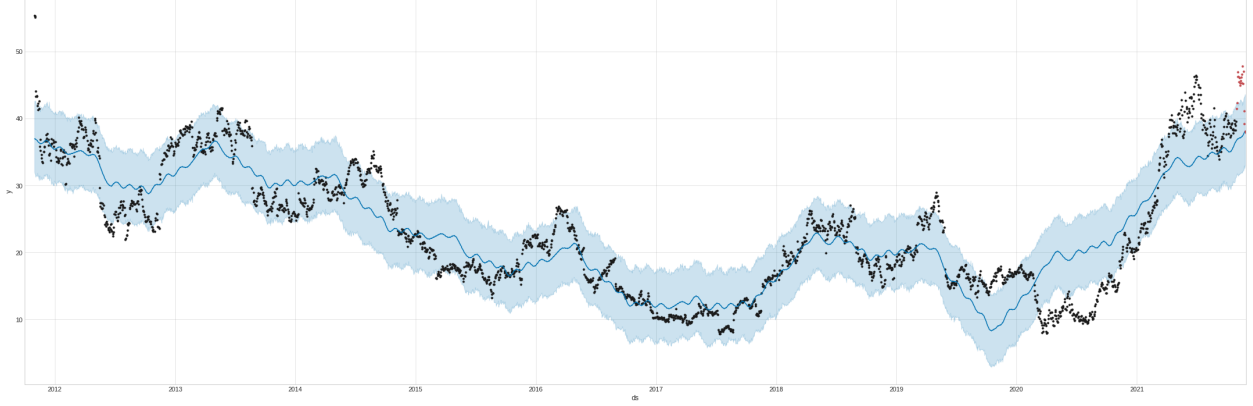


Figure 14: Detail of Prophet forecast for the ANF stock closing price.

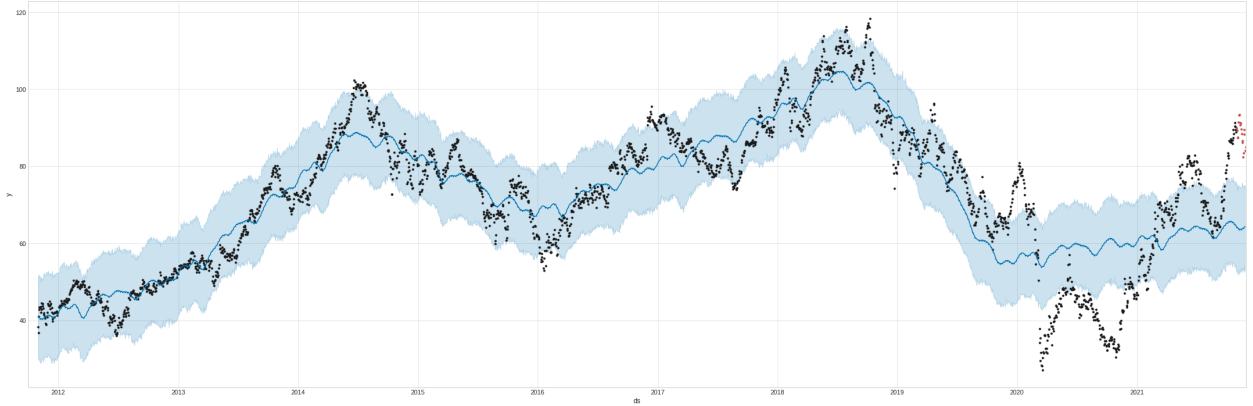


Figure 15: Detail of Prophet forecast for the EOG stock closing price.

Figs. 14 and 15 show for the ANF and EOG closing prices, respectively the historical data as black dots, the future actual data as the rightmost red dots, and the Prophet forecasts as a blue line.

It is clear that the model performs well in the first years but, when volatility increases (approximately from year 2020), the forecasts start to be clearly worse in the latest period as shown in detail in figs. 16 and 17.

Table 6: Error metrics of Prophet on ANF and EOG stock price prediction.

	Prophet				
	MSE	RMSE	MAE	MAPE	EVS
<i>ANF</i>	53.04	7.28	6.71	0.16	0.31
<i>EOG</i>	713.61	26.71	26.54	0.30	-0.03

Table 6 reports the corresponding error metrics calculated in the same time frame used for the ARIMA experiments. Clearly, the performances are unacceptable also in this case.

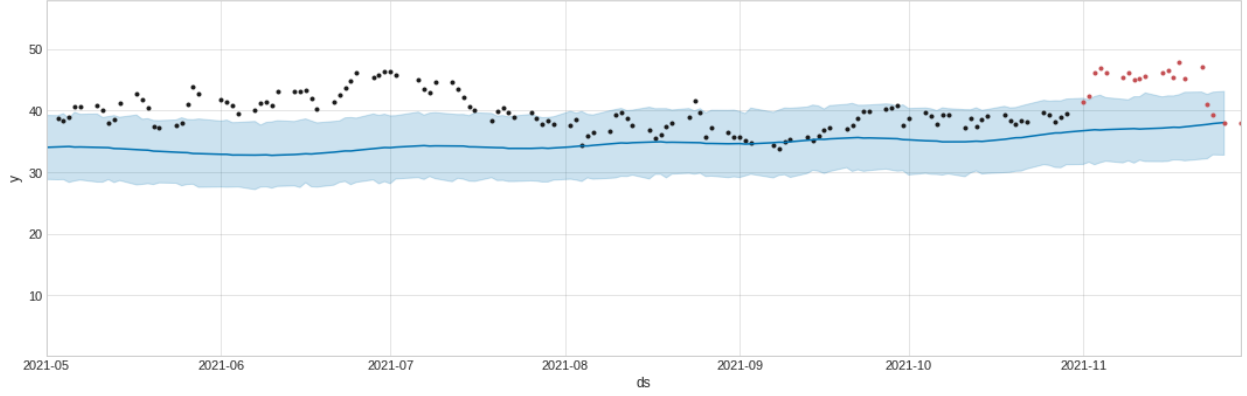


Figure 16: Detail of the last five months of Prophet forecast for the ANF stock closing price.

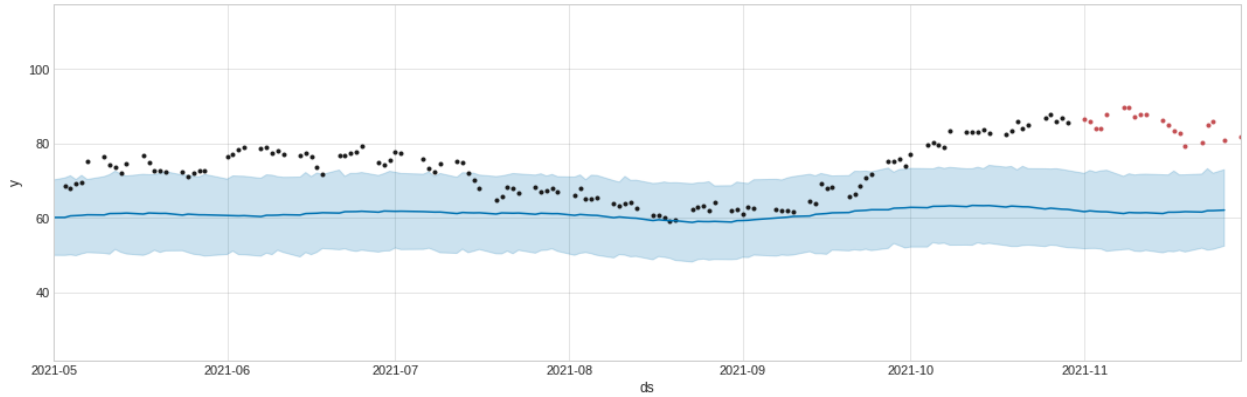


Figure 17: Detail of the last five months of Prophet forecast for the EOG stock closing price.

5 Forecasting with Deep Learning Methods

Compared with conventional artificial neural networks, deep neural networks are characterized by a higher number of neurons and hidden layers each of which, in principle, gives the network a greater ability to extract high-level features. This makes DNNs very efficient in solving nonlinear problems: in particular, when it comes to time series forecasting, DNNs can fill the gap left open by traditional statistical techniques such as the ones presented in Section 4, which often assume that the series are generated by linear processes and consequently may be inappropriate for most real-world problems that are overwhelmingly nonlinear. Indeed, works like Yao et al. [1999] and Hansen et al. [1999] (which focuses on time series prediction) show that neural network models often outperform conventional ARIMA models, and in particular Hansen et al. [1999] also show that neural networks outperform ARIMA in predicting the direction of stock movement, since are able to detect hidden patterns in the time series.

5.1 The Deep Neural Networks

In this subsection, we maintain the same forecasting objective of Section 4, i.e., $n = 30$ days following the training date, grouped however, in *batches* of $bs = 5$, corresponding to the working days in an open market week. We use four DNNs, one for each OHLC price, all with the same architecture. However, while the Auto-ARIMA detected that the optimal configuration for such algorithm was to generate a forecast based on the previous value (see Section 4.1, here we empirically found that the neural network performs better if its *input layer* is fed with the $t = 5$ previous values, i.e., the prices of the previous market week. In other words, to forecast the price of a day s , the input neurons will be presented to the prices of days $s - 1, \dots, s - 5$, respectively.

When building a neural network for applications like financial forecasting, one must find a compromise between *generalisation* and *convergence*. For example, hidden layers must not have too many nodes, since they may lead the DNNs to learn the training data without performing any generalization. Therefore, to find the geometry (number and

size of hidden layers) which minimizes the error on all the networks, we developed a python module that generates different network geometries in combination with the sklearn GridSearchCV²³ algorithm, which in turn tries to find the optimal combination of the hyperparameters (epochs, batch size, learning rate, optimizer employed) for each specific network.

The resulting optimal geometry has two hidden layers composed by $10 * t$ and $5 * t$ neurons respectively, as in Letteri et al. [2018] and Letteri et al. [2019a]. Finally, each network outputs its price prediction for the two considered stocks via a single neuron in the output layer.

To help reducing overfitting, we also applied a dropout of 0.2% on each of the two internal layers (Hinton et al. [2012]). In addition, to introduce non-linearity between layers, we used *ReLU* as the activation function, which performs better than a *tanh* or *sigmoid* functions (Krizhevsky et al. [2012]), despite the fact that the depth of the network consists of only a few internal layers.

To estimate the network learning performance during the training we use the *L1loss* function, which measures the mean absolute error (MAE) between each predicted value and the corresponding real one. The optimization algorithm used to minimize such loss function during the training is the *adaptive moment* (Adam), an extension of the *stochastic gradient descent* (SGD).

The trained networks are freely available and can be downloaded from our repository²⁴.

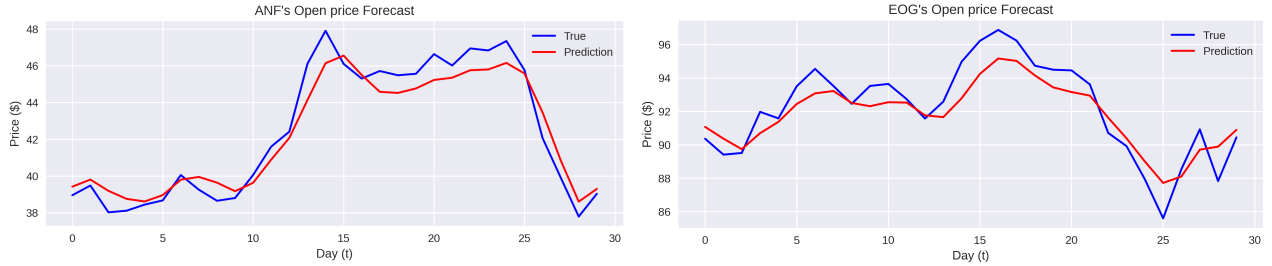


Figure 18: DNNs prediction of 30 day of ANF and EOG open prices (validation set)

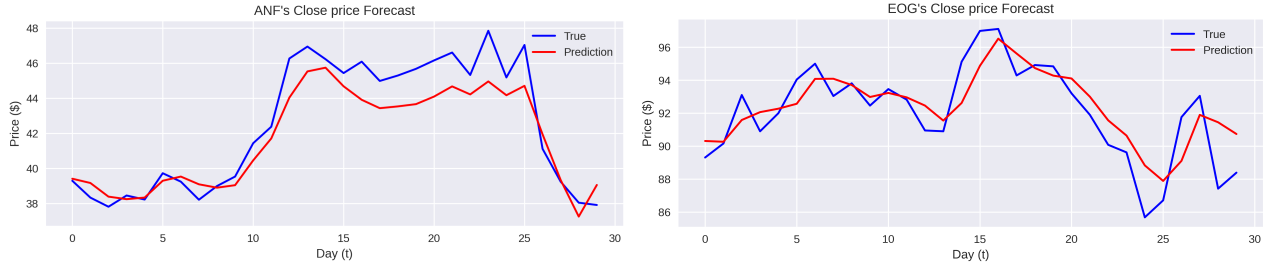


Figure 19: DNNs prediction of 30 day of ANF and EOG close prices (validation set)

Table 7: Error metrics of DNN on ANF and EOG stock *Close* price prediction (validation set)

	DNN				
	MSE	RMSE	MAE	MAPE	EVS
ANF	1.75	1.32	1.07	0.02	0.91
EOG	2.39	1.55	1.23	0.01	0.7

Figs. 18, 20, 21, and 19 show the DNN forecasts on the ANF and EOG open, high, low, and close prices, respectively, in the same 30-day time frame used for the experiments of the previous section. For the sake of brevity, we report only the corresponding error metrics about close prices in tab. 7. It is clear that the DNN performs better than the statistical methods shown in the previous section.

²³https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

²⁴<https://www.ivanletteri.it/dnnModels/>

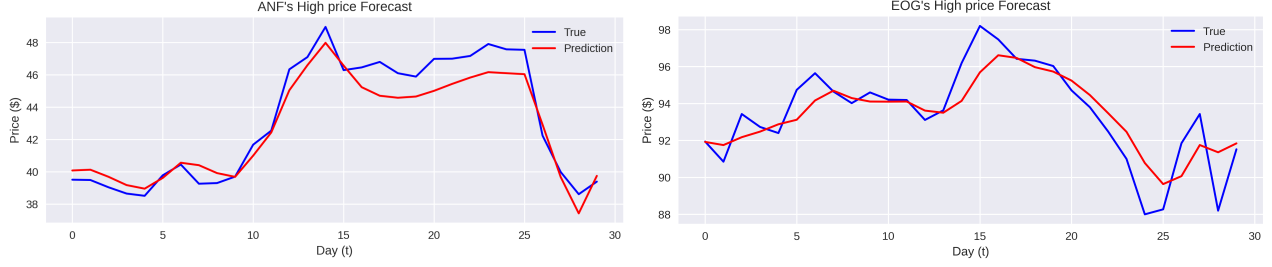


Figure 20: DNNs prediction of 30 day of ANF and EOG high prices (validation set)

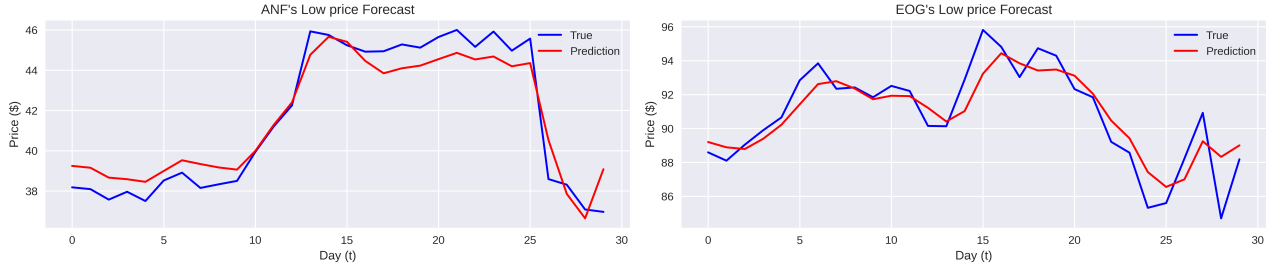


Figure 21: DNNs prediction of 30 day of ANF and EOG low prices (validation set)

5.2 DNN-Forwardtesting Experiment

After showing that DNNs are the best forecast model for our stock price dataset, we want to show the performances of a trading system that is based on such forecasts. A trading *strategy* tells the investor when to buy or sell shares in such a way that the sequence of these operations is profitable. Typically, discretionary traders base such strategy on the values of one or more technical indicators. System traders, which use algorithms to guide their trading, typically apply a rule-based approach, where rules are also based on a set of technical indicators. In both cases, such indicators are usually chosen by the trader using the so called *backtesting* technique, i.e., by considering the available historical data and choosing the indicator(s) so that the corresponding strategy would get the highest profit if applied on the past.

On the other hand, we propose a novel, alternative approach, which exploits the DNN forecasts and selects the indicators such that the corresponding trading strategy would get the highest profit on the *possible future* given by the forecasts. Our hypothesis is that the DNN forecasts may encode a deeper understanding of the past trends, i.e., we actually exploit the historical data in a way that the traditional approach would not be able to do.

In particular, the (algorithmic) trading strategy of our system is encoded in a set of *entry* and *exit trading rules* which are in turn based on the value of a single technical indicator chosen from a set of ten common technical indicators, i.e., Simple Moving Average (SMA), Exponential Moving Average (EMA), Moving Average Convergence Divergence (MACD), Bollinger Bands (BBs), Stochastic (ST), William %R (W%R), Momentum (MO), Relative Strength Index (RSI), Average true range (ATR), Price Oscillator (PO) (see Barnwal et al. [2019]), Triple Exponential Moving Average (TEMA, Tsantekidis et al. [2017]) and Average Directional Index (ADX). We also tested some further meaningful combinations of the above indicators, like in Prasad et al. [2022], and Hryshko and Downs [2004], such as ST+MO+MACD, PO+W%R, PO+RSI.

5.2.1 Results

The best indicator for ANF is the *Triple Exponential Moving Average*, whereas *Average Directional Index* is more suitable for EOG. In particular,

- the Triple Exponential Moving Average (TEMA) (Tsantekidis et al. [2017]) is generally used to make short-term and medium-term intraday trading decisions to enter long or short trades, depending on bullish or bearish signals. It was designed to smooth price fluctuations, making it easier to identify trends without the lag associated with moving averages.

- the Average Directional Index (ADX)²⁵ is used to determine the strength of a trend, and is accompanied by two further indicators: the negative directional indicator (-DI) and the positive directional indicator (+DI). As an example, when the +DI crosses above the -DI line, the rate of positive price change in the market is greater than the negative price change. If this happens when the ADX is above 25, it is a solid signal to place buy orders. Similarly, when the -DI crosses above the +DI line, it implies that the rate of negative price change in the market is greater than the positive price change. If this happens when the ADX is below 25, it is a solid signal to place sell orders.

ANF	
Entry	$((x^{(l)} < TEMA^{(l)}) \vee (x^{(h)} < TEMA^{(h)})) \wedge ((x^{(c)} < TEMA^{(c)}) \vee (x^{(o)} < TEMA^{(o)}))$
Exit	$((x^{(l)} > TEMA^{(l)}) \vee (x^{(h)} > TEMA^{(h)})) \wedge ((x^{(c)} > TEMA^{(c)}) \vee (x^{(o)} > TEMA^{(o)}))$
EOG	
Entry	$(+DI > -DI) \wedge (ADX > 25)$
Exit	$(-DI > +DI) \wedge (ADX > 25)$

Figure 22: Trading system rules.

The final trading rules, based on such indicators, are shown in fig. 22, where (o) , (h) , (l) , (c) refer to the OHLC prices, respectively, and x is the current (opening, highest, etc.) price.

Table 8: Performance of trading system on validation set with ANF and EOG stocks.

	Num. of Trades	Total Return (\$)	Expectancy Ratio	Sharpe Ratio	Sortino Ratio	Calmar Ratio
ANF	3	6.126	2.112	2.194	3.340	12.403
EOG	3	1.374	0.525	1.253	2.556	5.814

Then, we evaluated the profit derived from the application of such a strategy on the *real* data of the 30-day trading period following October 16, 2021, having as starting point a budget of \$100 reinvested in compounded mode. The results are shown in Table 8.

5.2.2 Baseline comparison

To compare our strategy with a baseline, we re-evaluated the same set of technical indicators through the traditional backtesting technique on the historical data for the 30 days *before* October 16, 2021, to see if it would result in different choices and maybe different profits.



Figure 23: Performance results on predicted ANF OHLC values to the left and EOG OHLC values to the right (N.B.: the candlestick graphs follow the asian notation).

²⁵https://en.wikipedia.org/wiki/Average_directional_movement_index

The details plotted using library Finlab²⁶ in fig. 23, shown that a trader using backtesting would choose ADX for the EOG share, as with DNN-forwardtesting technique, so the profit would be the same in this case. However, the TEMA indicator would not be chosen for the ANF share. Indeed, the most promising indicator, given the past 30 days of market, would be RSI with time period 5, overbought=70, oversell=30 as in fig. 24 but, if applied to the future, it would result in a lower profit.



Figure 24: RSI indicator on ANF close prices last 30 days of trainset (to the left) and 30 days in the future (to the right).

Table 9: RSI profit observing the latest 30 days price of ANF stocks.

	Num. of Trades	Total Return (\$)	Expectancy Ratio	Sharpe Ratio	Sortino Ratio	Calmar Ratio
ANF 30 days before	1	8.741	1.421	2.266	3.064	20.238
ANF 30 days after	1	-1.168	-1.1683	0.119	0.158	-0.935
EOG 30 days before	1	8.741	1.421	2.266	3.064	20.238
EOG 30 days after	1	-1.168	-1.1683	0.119	0.158	-0.935

6 Conclusion

In this paper, we propose a stock market trading system that exploits deep neural networks as part of its main components improving a previous work (Letteri et al. [2022b]).

In such a system, the trades are guided by the values of a pre-selected technical indicator, as usual in algorithmic trading. However, the novelty of the presented approach is in the indicator selection technique: traders usually make such a selection by *backtesting* the system on the historical market data and choosing the most profitable indicator with respect to the *known past*. On the other hand, in our approach, such most profitable indicator is chosen by *DNN-forwardtesting* it on the *probable future* predicted by a deep neural network trained on the historical data.

As discussed in the paper, neural networks outperform the most common statistical methods in stock price prediction: indeed, their predicted future allows to make a very accurate selection of the indicator to apply, which takes into account trends that would be very difficult to capture through backtesting.

To validate this claim, we applied our methodology on two very different assets with medium volatility, and the results show that our DNN forwardtesting-based trading system achieves a profit that is equal or higher than the one of a traditional backtesting-based trading system.

Given the promising potentials of this approach, we will further test its reliability on other stock markets using different data, such as cryptocurrencies or defi-tokens, also varying the timeframes for day trading and scalping activities.

²⁶<https://pypi.org/project/finlab-crypto/>

In the near future we plan to explore data analysis following a proven process and feature selection strategy (see Letteri et al. [2020b], Letteri et al. [2019b]). The new datasets will certainly require a balancing (e.g., buy, sell and hold trades). This balancing will take place according to the algorithm called *Generative 1 Nearest Neighbours* (Letteri et al. [2020c]) and targeted oversampling (Letteri et al. [2021]).

In the future work, we consider to analyze DNNs considering them some black-box decision-making systems, which inherently introduce the risk of impacting aspects such as ethics (Dyoub et al. [2021a]) further when spread to Multi Agent System (Dyoub et al. [2021b]), and we will analyze them from a Logic-based Machine Learning perspective (Dyoub et al. [2020]).

References

- Eugene F. Fama. The behavior of stock-market prices. *The Journal of Business*, 38(1):34–105, 1965. ISSN 00219398, 15375374. URL <http://www.jstor.org/stable/2350752>.
- Ayodele Ariyo Adebisi, Aderemi Oluyinka Adewumi, and Charles Korede Ayo. Comparison of arima and artificial neural networks models for stock price prediction. *Journal of Applied Mathematics*, 2014:1–7, 2014. URL <https://EconPapers.repec.org/RePEc:hin:jnljam:614342>.
- Thomas Lux and Taisei Kaizoji. Forecasting volatility and volume in the tokyo stock market: Long memory, fractality and regime switching. *Journal of Economic Dynamics and Control*, 31(6):1808–1843, 2007. ISSN 0165-1889. doi:<https://doi.org/10.1016/j.jedc.2007.01.010>. URL <https://www.sciencedirect.com/science/article/pii/S0165188907000024>. Tenth Workshop on Economic Heterogeneous Interacting Agents.
- Ivan Letteri, Giuseppe Della Penna, and Giovanni De Gasperis. Botnet detection in software defined networks by deep learning techniques. 11161:49–62, 2018. doi:10.1007/978-3-030-01689-0_4.
- Gonzalo Marín, Pedro Caasas, and Germán Capdehourat. Deepmal - deep learning models for malware traffic detection and classification. In Peter Haber, Thomas Lampoltshammer, Manfred Mayr, and Kathrin Plankensteiner, editors, *Data Science – Analytics and Applications*, pages 105–112, Wiesbaden, 2021. Springer Fachmedien Wiesbaden. ISBN 978-3-658-32182-6.
- Ivan Letteri, Giuseppe Della Penna, and Giovanni De Gasperis. Security in the internet of things: botnet detection in software-defined networks by deep learning techniques. *Int. J. High Perform. Comput. Netw.*, 15(3/4):170–182, 2019a. doi:10.1504/IJHPCN.2019.106095.
- Ivan Letteri, Antonio Di Cecco, and Giuseppe Della Penna. New optimization approaches in malware traffic analysis. In Giuseppe Nicosia, Varun Ojha, Emanuele La Malfa, Gabriele La Malfa, Giorgio Jansen, Panos M. Pardalos, Giovanni Giuffrida, and Renato Umeton, editors, *Machine Learning, Optimization, and Data Science*, pages 57–68, Cham, 2022a. Springer International Publishing. ISBN 978-3-030-95467-3. doi:10.1007/978-3-030-95467-3_4.
- Soniya, Sandeep Paul, and Lotika Singh. A review on advances in deep learning. *2015 IEEE Workshop on Computational Intelligence: Theories, Applications and Future Directions (WCI)*, pages 1–6, 2015.
- Min-Yuh Day and Chia-Chou Lee. Deep learning for financial sentiment analysis on finance news providers. *2016 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*, pages 1127–1134, 2016.
- Huiwen Wang, Wenyang Huang, and Shanshan Wang. Forecasting open-high-low-close data contained in candlestick chart, 2021.
- Can Yang, Junjie Zhai, and Helong Li. On the bound of cumulative return in trading series and the verification using technical trading rules, 2020. URL <https://arxiv.org/abs/2005.13974>.
- Thomas Mikosch and Cătălin Stărică. Limit theory for the sample autocorrelations and extremes of a GARCH (1,1) process. *The Annals of Statistics*, 28(5):1427 – 1451, 2000. doi:10.1214/aos/1015957401. URL <https://doi.org/10.1214/aos/1015957401>.
- Benjamin H. Hoerlein. Modeling volatility of financial time series using arc length. 2017.
- Gábor Petneházi and József Gáll. Exploring the predictability of range-based volatility estimators using rnns, 2018. URL <https://arxiv.org/abs/1803.07152>.
- Jaydip Sen, Sidra Mehtab, and Abhishek Dutta. Volatility modeling of stocks from selected sectors of the indian economy using GARCH. In *2021 Asian Conference on Innovation in Technology (ASIANCON)*. IEEE, aug 2021. doi:10.1109/asiancon51346.2021.9544977.
- Dennis Yang and Qiang Zhang. Drift-independent volatility estimation based on high, low, open, and close prices. *The Journal of Business*, 73(3):477–492, 2000. ISSN 00219398, 15375374. URL <http://www.jstor.org/stable/10.1086/209650>.

- Richard J. Martin. Design and analysis of momentum trading strategies. 2021. URL <https://EconPapers.repec.org/RePEc:arx:papers:2101.01006>.
- Peng Huang and Tianxiang Wang. On the profitability of optimal mean reversion trading strategies, 2016.
- Robert J. Elliott, John Van Der Hoek, and William P. Malcolm. Pairs trading. *Quantitative Finance*, 5(3):271–276, 2005. doi:10.1080/14697680500149370.
- Marco Avellaneda and Jeong-Hyun Lee. Statistical arbitrage in the us equities market. *Quantitative Finance*, 10(7):761–782, 2010. doi:10.1080/14697680903124632.
- Ivan Letteri, Antonio Di Cecco, Abeer Dyoub, and Giuseppe Della Penna. A novel resampling technique for imbalanced dataset optimization. *CoRR*, abs/2012.15231, 2020a. URL <https://arxiv.org/abs/2012.15231>.
- David Arthur and Sergei Vassilvitskii. K-means++: The advantages of careful seeding. volume 8, pages 1027–1035, 01 2007. doi:10.1145/1283383.1283494.
- F. Black. Studies of stock price volatility changes. 3002:177–181, 1976.
- A.C. Linke, L.E. Mash, C.H. Fong, M.K. Kinneer, J.S. Kohli, M. Wilkinson, R. Tung, R.J. Jao Keehn, R.A. Carper, I. Fishman, and R.-A. Müller. Dynamic time warping outperforms pearson correlation in detecting atypical functional connectivity in autism spectrum disorders. *NeuroImage*, 223:117383, 2020. ISSN 1053-8119. doi:<https://doi.org/10.1016/j.neuroimage.2020.117383>.
- Jamie Marquez. Time series analysis : James d. hamilton, 1994, (princeton university press, princeton, nj). *International Journal of Forecasting*, 11(3):494–495, September 1995.
- H. Akaike. A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6):716–723, 1974. doi:10.1109/TAC.1974.1100705.
- Rob Hyndman and Yeasmin Khandakar. Automatic time series forecasting: The forecast package for r. *Journal of Statistical Software*, 26, 07 2008. doi:10.18637/jss.v027.i03.
- P. Stoica and Y. Selen. Model-order selection: a review of information criterion rules. *IEEE Signal Processing Magazine*, 21(4):36–47, 2004. doi:10.1109/MSP.2004.1311138.
- Oleh Danyliv, Bruce Bland, and Alexandre Argenson. Random walk model from the point of view of algorithmic trading, 2019.
- Emir Žunić, Kemal Korjenić, Kerim Hodžić, and Dženana Đonko. Application of facebook’s prophet algorithm for successful sales forecasting based on real-world data. *International Journal of Computer Science and Information Technology*, 12(2):23–36, Apr 2020. ISSN 0975-4660. doi:10.5121/ijcsit.2020.12203.
- Facebook Open Source. Prophet github pages. <https://facebook.github.io/prophet/>, 2022.
- Jingtao Yao, Chew Lim Tan, and Hean-Lee Poh. Neural networks for technical analysis: a study on klci. *International journal of theoretical and applied finance*, 2(02):221–241, 1999.
- James V Hansen, James B McDonald, and Ray D Nelson. Time series prediction with genetic-algorithm designed neural networks: An empirical comparison with modern statistical models. *Computational Intelligence*, 15(3):171–184, 1999.
- Geoffrey E. Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *CoRR*, abs/1207.0580, 2012. URL <http://arxiv.org/abs/1207.0580>.
- Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Proceedings of the 25th International Conference on Neural Information Processing Systems - Volume 1*, NIPS’12, page 1097–1105, Red Hook, NY, USA, 2012. Curran Associates Inc.
- Avinash Barnwal, Hari Pad Bharti, Aasim Ali, and Vishal Singh. Stacking with neural network for cryptocurrency investment. In *2019 New York Scientific Data Summit (NYSDS)*, pages 1–5, 2019. doi:10.1109/NYSDS.2019.8909804.
- Avraam Tsantekidis, Nikolaos Passalis, Anastasios Tefas, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis. Forecasting stock prices from the limit order book using convolutional neural networks. In *2017 IEEE 19th Conference on Business Informatics (CBI)*, volume 01, pages 7–12, 2017. doi:10.1109/CBI.2017.23.
- P. Shanmukh Kali Prasad, Vadlamani Madhav, Ramanuj Lal, and Vadlamani Ravi. Optimal technical indicator-based trading strategies using nsga-ii, 2022.
- A. Hryshko and T. Downs. System for foreign exchange trading using genetic algorithms and reinforcement learning. *International Journal of Systems Science*, 35(13-14):763–774, 2004. doi:10.1080/00207720412331303697.

- Ivan Letteri, Giuseppe Della Penna, Giovanni De Gasperis, and Abeer Dyoub. A stock trading system for a medium volatile asset using multi layer perceptron, 2022b.
- Ivan Letteri, Giuseppe Della Penna, Luca Di Vita, and Maria Teresa Grifa. Mta-kdd'19: A dataset for malware traffic detection. 2597:153–165, 2020b. URL <http://ceur-ws.org/Vol-2597/paper-14.pdf>.
- Ivan Letteri, Giuseppe Della Penna, and Pasquale Caianiello. Feature selection strategies for HTTP botnet traffic detection. pages 202–210, 2019b. doi:10.1109/EuroSPW.2019.00029.
- Ivan Letteri, Antonio Di Cecco, and Giuseppe Della Penna. Dataset optimization strategies for malware traffic detection. *CoRR*, abs/2009.11347, 2020c. URL <https://arxiv.org/abs/2009.11347>.
- Ivan Letteri, Antonio Di Cecco, Abeer Dyoub, and Giuseppe Della Penna. Imbalanced dataset optimization with new resampling techniques. In Kohei Arai, editor, *Intelligent Systems and Applications - Proceedings of the 2021 Intelligent Systems Conference, IntelliSys 2021, Amsterdam, The Netherlands, 2-3 September, 2021, Volume 2*, volume 295 of *Lecture Notes in Networks and Systems*, pages 199–215. Springer, 2021. doi:10.1007/978-3-030-82196-8_15. URL https://doi.org/10.1007/978-3-030-82196-8_15.
- Abeer Dyoub, Stefania Costantini, Francesca A. Lisi, and Ivan Letteri. Ethical monitoring and evaluation of dialogues with a mas. 3002:158–172, 2021a.
- Abeer Dyoub, Stefania Costantini, Ivan Letteri, and Francesca A. Lisi. A logic-based multi-agent system for ethical monitoring and evaluation of dialogues. In Andrea Formisano, Yanhong Annie Liu, Bart Bogaerts, Alex Brik, Verónica Dahl, Carmine Dodaro, Paul Fodor, Gian Luca Pozzato, Joost Vennekens, and Neng-Fa Zhou, editors, *Proceedings 37th International Conference on Logic Programming (Technical Communications), ICLP Technical Communications 2021, Porto (virtual event), 20-27th September 2021*, volume 345 of *EPTCS*, pages 182–188, 2021b. doi:10.4204/EPTCS.345.32. URL <https://doi.org/10.4204/EPTCS.345.32>.
- Abeer Dyoub, Stefania Costantini, Francesca Alessandra Lisi, and Ivan Letteri. Logic-based machine learning for transparent ethical agents. In Francesco Calimeri, Simona Perri, and Ester Zumpano, editors, *Proceedings of the 35th Italian Conference on Computational Logic - CILC 2020, Rende, Italy, October 13-15, 2020*, volume 2710 of *CEUR Workshop Proceedings*, pages 169–183. CEUR-WS.org, 2020. URL <http://ceur-ws.org/Vol-2710/paper11.pdf>.